ASSIGNMENT-13.5

HT.NO-2403A52L06

BATCH-50

QUESTION:

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

Week7 -

Friday

• Identify code smells and inefficiencies in legacy Python scripts.

• Use AI-assisted coding tools to refactor for readability, maintainability, and performance.

• Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Global Variables) • Task: Use AI to eliminate unnecessary global variables from the code.

• Instructions:

o Identify global variables used across functions. o Refactor the code to pass values using function parameters. o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1 def
calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
```

• Expected Output:

o Refactored version passing rate as a parameter or using a configuration structure.

Task Description #2 : (Refactoring Deeply Nested Conditionals) •

Task: Use AI to refactor deeply nested if–elif–else logic into a

cleaner structure.

• Focus Areas: o

Readability o Logical

simplification o

Maintainability

Legacy Code:

score = 78 if score

>= 90:

print("Excellent")

else: if score >=

75: print("Very

Good")

else: if score

>= 60:

print("Good")

else:

print("Needs Improvement")

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based

approach.

Task 3 (Refactoring Repeated File Handling Code) • Task:

Use AI to refactor repeated file open/read/close logic.

• Focus Areas: o

DRY principle o

Context managers o

Function reuse

Legacy Code:

f = open("data1.txt") print(f.read())

f.close() f =

open("data2.txt")

print(f.read())

f.close()

Expected Outcome:

o Reusable function using with open() and parameters.

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear

searches using appropriate data structures.

- Focus Areas: o Time complexity o

Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"] name =

input("Enter username: ") found = False for u in users:

if u == name: found = True print("Access Granted" if

found else "Access Denied") Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Task 5 (Refactoring Procedural Code into OOP Design) •

Task: Use AI to refactor procedural code into a class-based

design.

- Focus Areas: o Object-Oriented principles o Encapsulation

  Legacy Code: salary = 50000 tax = salary * 0.2 net = salary -

  tax print(net)

Expected Outcome: o A class like

EmployeeSalaryCalculator with methods and attributes.

Task 6 (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling

  large data.

• Focus Areas:

o Algorithmic optimization o Use

of built-in functions

Legacy Code:

total = 0 for i in range(1,

1000000): if i % 2 == 0:

total += i print(total)

Expected Outcome:

o Optimized logic using

mathematical formulas or

comprehensions, with time

comparison. Task 7 (Removing Hidden

Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring o

Predictability

Legacy Code:

data = [] def

add_item(x):

data.append(x)

add_item(10)

add_item(20)

print(data)

Expected Outcome:

o Refactored function returning values

instead of mutating globals.

Task 8 (Refactoring Complex Input Validation Logic) • Task:

Use AI to simplify and modularize complex validation rules.

• Focus Areas: o Readability o

Testability Legacy Code: password =

input("Enter password: ") if

len(password) >= 8: if any(c.isdigit()

for c in password): if any(c.isupper()

for c in password):

print("Valid Password") else:

print("Must contain uppercase")

else:

print("Must contain digit") else:

print("Password too short")

Expected Outcome: o Separate validation functions

with clear responsibility

GENERATED CODE:

Lab 13: Code Refactoring – Improving Legacy Code with AI

---------- Task 1: Removing Global Variables ----------

def calculate_interest(amount, rate):

return amount * rate

---------- Task 2: Refactoring Nested Conditionals ----------

def grade_message(score): if score >= 90:

return "Excellent"

if score >= 75:

return "Very Good" if

score >= 60: return

"Good" return "Needs

Improvement"

```
# ---------------- Task 3: Repeated File Handling ----------------
def read_file(filename):     with open(filename) as f:
        return f.read()

# ---------------- Task 4: Optimizing Search Logic ---------------- def
check_user_access(name, users={"admin", "guest", "editor", "viewer"}):
   return "Access Granted" if name in users else "Access Denied"
# ---------------- Task 5: Procedural → OOP ----------------
class EmployeeSalaryCalculator:     def __init__(self,
salary, tax_rate=0.2):
        self.salary = salary
self.tax_rate = tax_rate

    def net_salary(self):         tax =
self.salary * self.tax_rate
return self.salary - tax

# ---------------- Task 6: Performance Optimization ----------------
def sum_even_numbers(limit=1_000_000):
    last_even = limit if limit % 2 == 0 else limit - 1
return (last_even // 2) * (last_even + 2) // 2

# ---------------- Task 7: Removing Hidden Side Effects ----------------
def add_item(data, x):     return data + [x]

# ---------------- Task 8: Simplifying Input Validation ----------------
- def is_long_enough(password):     return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)
```

```
def has_upper(password):
   return any(c.isupper() for c in password)
def validate_password(password):
if not is_long_enough(password):
return "Password too short"     if not
has_digit(password):         return
"Must contain digit"     if not
has_upper(password):         return
"Must contain uppercase"     return
"Valid Password"


# ---------------- Menu System ----------------
def main():     while True:
        print("\n--- Lab 13 Refactoring Menu ---")
print("1. Calculate Interest")         print("2.
Grade Message")         print("3. Read File")
print("4. Check User Access")         print("5.
Employee Salary Calculator")         print("6.
Sum Even Numbers")         print("7. Add Item
(No Side Effects)")         print("8. Validate
Password")         print("9. Exit")


        choice = input("Choose a task (1-9): ")


        if choice == "1":
            print(calculate_interest(1000, 0.1))
 elif choice == "2":
            score = int(input("Enter score: "))
print(grade_message(score))         elif choice == "3":
```

```
        filename = input("Enter filename: ")
print(read_file(filename))        elif choice
== "4":
        name = input("Enter username: ")
print(check_user_access(name))        elif
choice == "5":
        salary = float(input("Enter salary: "))
emp = EmployeeSalaryCalculator(salary)
print("Net Salary:", emp.net_salary())        elif
choice == "6":
        limit = int(input("Enter limit (default 1,000,000): ") or 1_000_000)
print("Sum of even numbers:", sum_even_numbers(limit))        elif
choice == "7":
        data = []            data = add_item(data, int(input("Enter
first item:")))            data = add_item(data, int(input("Enter
second item:")))            print("Final List:", data)        elif
choice == "8":
        password = input("Enter password: ")
print(validate_password(password))        elif
choice == "9":            print("Exiting...")
break        else:
        print("Invalid choice, try again.")


if __name__ == "__main__":
    main()
```

OUTPUT:

```
PROBLEMS 533   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY RESULTS                Python

sanjaykumar@perumandlas-MacBook-Air AI - assiant %  source "/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/acti
vate"
"/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/python" "/Users/sanjaykumar/Documents/AI - assiant/ass-13.5.py"
(.venv) sanjaykumar@perumandlas-MacBook-Air AI - assiant % "/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/pyth
on" "/Users/sanjaykumar/Documents/AI - assiant/ass-13.5.py"

--- Lab 13 Refactoring Menu ---
1. Calculate Interest
2. Grade Message
3. Read File
4. Check User Access
5. Employee Salary Calculator
6. Sum Even Numbers
7. Add Item (No Side Effects)
8. Validate Password
9. Exit
Choose a task (1-9): 1
100.0
```

```
Choose a task (1-9): 2
Enter score: 55
Needs Improvement
```

```
Choose a task (1-9): 3
Enter filename: ai.txt
h
h
ggf
df
df
fd
g
g
dg
sgd
g
```

```
Choose a task (1-9): 4
Enter username: aa
Access Denied
```

```
Choose a task (1-9): 5
Enter salary: 50000
Net Salary: 40000.0
```

```
Choose a task (1-9): 6
Enter limit (default 1,000,000): 1000000
Sum of even numbers: 250000500000
```

```
Choose a task (1-9): 7
Enter first item: 2
Enter second item: 6
Final List: [2, 6]
```

```
Choose a task (1-9): 8
Enter password: akhi
Password too short
```